Experiment 06: Docker Images and Deploy Containerized Applications using Docker

Experiment No.	06
Mapped CO	CO4
Aim	To create Docker images and deploy containerized applications using Docker.
Tools / Software	Docker Engine 24 or later; Ubuntu 22.04+ / Windows 11 with WSL2; Docker Hub account; A sample web application

Theory

In DevOps, application deployment should be repeatable, portable and easy to verify. Docker supports this requirement by packaging an application, runtime, dependencies and configuration into a deployable Docker image. The same image can be executed as a container on any compatible host.

A Docker image is a versioned, read-only package created from a Dockerfile. A container is the running instance of that image. For deployment, images are usually tagged with version names, pushed to a registry such as Docker Hub, pulled on a target machine and executed using docker run with suitable port mapping and restart options.

Common Dockerfile instructions such as FROM, WORKDIR, COPY, RUN, EXPOSE and CMD define how the image is built and how the application starts. During deployment, options such as -d, --name, -p and --restart help run the application in background mode, expose it to the host and keep it available after service restarts.

Procedure

1. Install Docker Engine and verify the installation using docker --version and docker info.
2. Create a project directory named docker-deployment-demo and place the sample application files inside it.
3. Create app.py with a simple Flask web route and create requirements.txt with the required dependency.
4. Write a Dockerfile defining the base image, working directory, dependency installation, exposed port and startup command.
5. Build a versioned Docker image using docker build and tag it as mywebapp:1.0.
6. Run the image locally as a container with port mapping and verify the output in a browser or using curl.
7. Tag the image for Docker Hub or registry-based deployment and optionally push it to the registry.
8. Deploy the image as a detached container using docker run with --name, -p and --restart options.
9. Verify deployment using docker ps, docker logs and browser/curl output.
10. Modify the application, build a new version such as mywebapp:2.0 and redeploy by replacing the old container.
11. Stop and remove containers/images after completion to clean up resources.

Sample Code / Commands

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello from Docker deployed application!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build, deploy, verify and redeploy commands

```
docker --version
docker build -t mywebapp:1.0 .
docker run -d --name mywebapp -p 8080:5000 --restart unless-stopped mywebapp:1.0
docker ps
curl http://localhost:8080
docker logs mywebapp
docker tag mywebapp:1.0 <dockerhub-user>/mywebapp:1.0
docker push <dockerhub-user>/mywebapp:1.0
docker stop mywebapp && docker rm mywebapp
docker build -t mywebapp:2.0 .
docker run -d --name mywebapp -p 8080:5000 --restart unless-stopped mywebapp:2.0
```

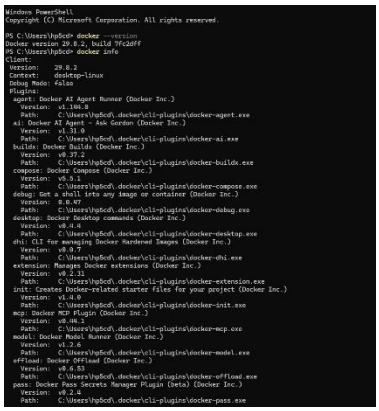
Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Docker image creation, container deployment and verification. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Gargee Warke	_65_____	_b2_____	_____

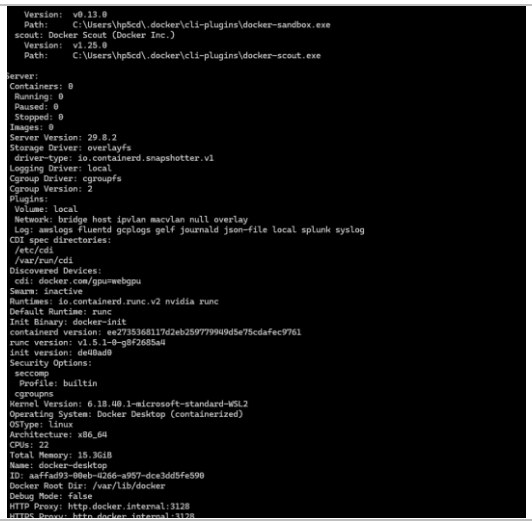
1. Docker Installation and Project Setup Output

Paste screenshot/output showing Docker version, Docker info and the created project directory.



2. Application Source and Dockerfile Output

Paste screenshot/output showing app.py, requirements.txt and Dockerfile contents.



3. Docker Image Build and Tag Output

Paste screenshot/output showing successful docker build command and image tag mywebapp:1.0.

```
PS C:\Users\hp5cd> mkdir docker-flask-demo


    Directory: C:\Users\hp5cd


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10-10-2026     16:18                docker-flask-demo


PS C:\Users\hp5cd> cd docker-flask-demo
PS C:\Users\hp5cd\docker-flask-demo> notepad app.py
PS C:\Users\hp5cd\docker-flask-demo> notepad requirements.txt
PS C:\Users\hp5cd\docker-flask-demo> notepad Dockerfile
PS C:\Users\hp5cd\docker-flask-demo> dir


    Directory: C:\Users\hp5cd\docker-flask-demo


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10-10-2026     16:25            232 app.py
-a----        10-10-2026     16:30            175 Dockerfile
-a----        10-10-2026     16:27              0 Dockerfile.txt
-a----        10-10-2026     16:26             12 requirements.txt
```

4. Container Deployment and Port Mapping Output

Paste screenshot/output showing docker run command, container name, restart policy and port mapping.

```
PS C:\Users\hp5cd\docker-flask-demo> docker build -t myapp:1.0 .
[+] Building 18.2s (10/10) FINISHED                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                  0.1s
 => => transferring dockerfile: 214B                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                   3.9s
 => [internal] load .dockerignore                                                     0.0s
 => => transferring context: 2B                                                       0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c6  6.8s
 => => resolve docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c6  0.0s
 => => sha256:bbc5a12237474ec0b23b5051260d56d6436a24d70d9cad961adebe7a037ed813 249B / 249B  0.3s
 => => sha256:6912e23eb44595b44dc6d9882b34f305ad6f6e9667cec70b13e5dbc4f8cd9 12.12MB / 12.12MB  5.4s
 => => sha256:75b6a36c64a1cf3520fc4b4aae4739d5caa8a6a05c27835a920a61784c9c32f 1.29MB / 1.29MB  1.2s
 => => sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e 29.84MB / 29.84MB  5.1s
 => => extracting sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c3201a74299113e05e  0.9s
 => => extracting sha256:75b6a36c64a1cf3520fc4b4aae4739d5caa8a6a05c27835a920a61784c9c32f8  0.1s
 => => extracting sha256:6912e23eb44595b44dc6d9882b34f305ad6f6e9667cec70b13e5dbc4f8cd97f4  0.5s
 => => extracting sha256:bbc5a12237474ec0b23b5051260d56d6436a24d70d9cad961adebe7a037ed813  0.0s
 => [internal] load build context                                                     0.1s
 => => transferring context: 575B                                                     0.0s
 => [2/5] WORKDIR /app                                                                0.1s
 => [3/5] COPY requirements.txt .                                                     0.1s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                          3.8s
 => [5/5] COPY . .                                                                    0.1s
 => exporting to image                                                                3.1s
 => => exporting layers                                                               0.9s
 => => exporting manifest sha256:6aeb888f465e198d10fcc71b52ed99631adcba23bef36fea19effa62d060  0.0s
 => => exporting config sha256:64e82d31ccb084186e8e2ef30fbbfbec2bf73d8225fe3691529364834d17d  0.0s
 => => exporting attestation manifest sha256:6f36ad962706091ca742ae0a5c860627fb611ffd02e3382f  0.0s
 => => exporting manifest list sha256:d8ab5770f3b8204c577a80b734c426fa057def172e8dd6365acb775  0.0s
 => => naming to docker.io/library/myapp:1.0                                          0.0s
 => => unpacking to docker.io/library/myapp:1.0                                       2.0s
```

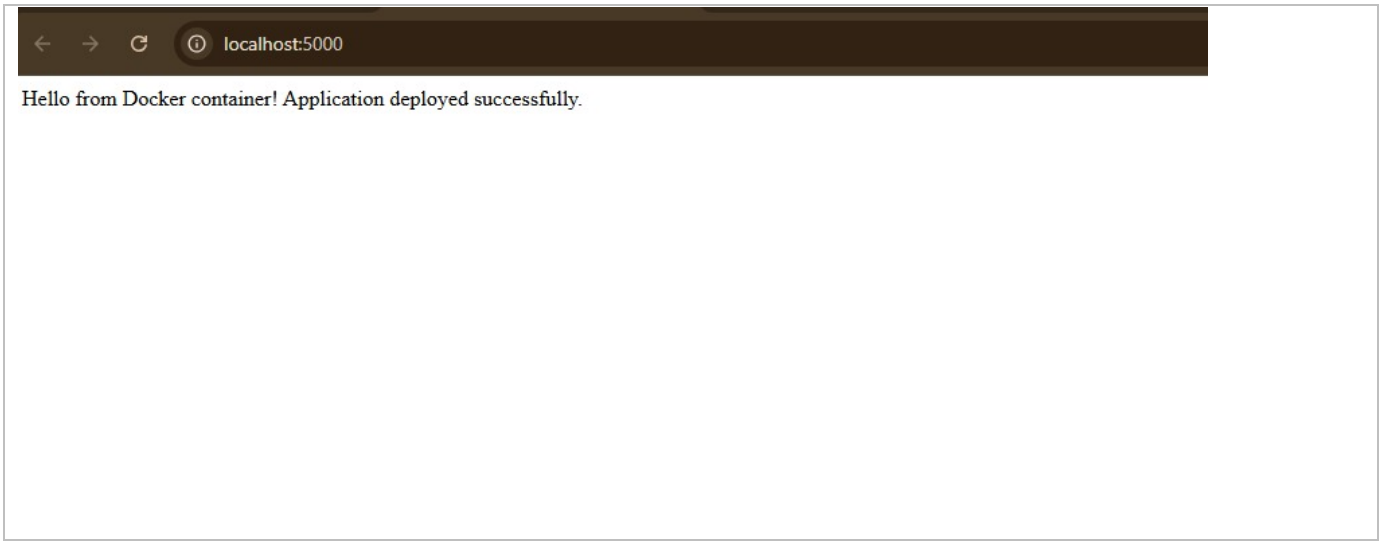
5. Application Access and Logs Output

Paste screenshot/output showing browser/curl response, docker ps and docker logs output.

```
PS C:\Users\hp5cd\docker-flask-demo> docker images
                                                                  i Info →  U In Use
IMAGE       ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:1.0   d8ab5770f3b8       198MB         48.3MB
PS C:\Users\hp5cd\docker-flask-demo> docker run -d -p 5000:5000 --name myapp-container myapp:1.0
93898542bc9cdc50f6e7f22cee8368ff1445ae2ef536460040f6fea1d119b9225
PS C:\Users\hp5cd\docker-flask-demo> docker ps
CONTAINER ID   IMAGE         COMMAND           CREATED          STATUS          PORTS                                         NAMES
93898542bc9c   myapp:1.0   "python app.py"   30 seconds ago   Up 30 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   myapp-container
PS C:\Users\hp5cd\docker-flask-demo> docker logs myapp-container
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [10/Oct/2026 11:06:52] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [10/Oct/2026 11:06:52] "GET /favicon.ico HTTP/1.1" 404 -
PS C:\Users\hp5cd\docker-flask-demo> docker stop myapp-container
myapp-container
PS C:\Users\hp5cd\docker-flask-demo> docker rm myapp-container
myapp-container
PS C:\Users\hp5cd\docker-flask-demo> docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\Users\hp5cd\docker-flask-demo>
```

6. Docker Hub Push / Pull or Redeployment Output

Paste screenshot/output showing image push/pull or updated version deployment such as mywebapp:2.0.



GitHub Repository / Lab Folder Link:

_____

Docker Hub Repository Link (Optional):

_____

Final Observation Statement:

```
Write 2-3 lines summarizing the result of the experiment.
```

Expected Output

Docker should be installed and working. The sample web application and Dockerfile should be created successfully. A versioned Docker image should be built, deployed as a container with port mapping, and verified through browser/curl output. Container status and logs should be visible using docker ps and docker logs. Optional registry push/pull or updated redeployment should also be demonstrated.

Conclusion

Docker images were created and used to deploy a containerized web application. The experiment demonstrated image building, tagging, container deployment, port mapping, log inspection and redeployment of an updated version.

Viva / Review Questions

1. What is the difference between a Docker image and a Docker container?
2. Why are version tags useful during Docker-based deployment?
3. Explain the role of port mapping in docker run.
4. What is the purpose of the --restart option during container deployment?

Marks: ____ / 10 **Date: _____** **Signature of Faculty: _____**